Manipulación de Cadenas y Punteros en C/C++: Control de Acceso por Edad Operando Directamente en Memoria RAM

Aramis Demaria

E.E.S.T. N.º 1 “Eduardo Ader” – Vicente López

Laboratorio de Programación (LPR) – 5.º Año 3.º A-B — Grupo A

Prof. Mansilla Muñoz York Elías

2026 — Grupo A

Resumen

Se desarrolla un control de acceso que manipula la variable de edad mediante un puntero, evidenciando la organización de la memoria RAM en casilleros con dirección hexadecimal. Se analizan la definición de puntero y desreferenciación, el operador de dirección (&), el de desreferenciación (*) y la diferencia entre imprimir *p* y **p*. Las pruebas con edades de 16 y 21 años muestran la bifurcación APROBADO y RECHAZADO y la dirección física variable. El informe responde la pregunta introductoria de la guía sobre qué es un puntero y la desreferenciación y las tres preguntas del cuestionario técnico: diferencia entre operadores, error crítico al guardar entrada sin desreferenciar y uso de notación hexadecimal para direcciones.

Palabras clave: puntero, desreferenciación, dirección de memoria, RAM, hexadecimal, control de acceso

Abstract

An access control that manipulates the age variable through a pointer is developed, evidencing RAM organization into cells with hexadecimal addresses. The definition of pointer and dereferencing, the address-of (&) and dereference (*) operators, and the difference between printing *p* and **p* are analyzed. Tests with ages 16 and 21 show the APPROVED and REJECTED branches and the variable physical address. The report answers the guide's introductory question on what a pointer and dereferencing are and the three technical questionnaire items.

Keywords: pointer, dereferencing, memory address, RAM, hexadecimal, access control

Introducción

Para comprender C y C++ en bajo nivel es necesario visualizar la memoria RAM como casilleros numerados con dirección única. La Actividad 4 solicita un sistema mínimo de control: leer la edad mediante puntero y decidir el acceso para mayores de 18 años,

mostrando valor y dirección hexadecimal. Los objetivos son comprender qué es un puntero y la desreferenciación y responder las tres preguntas del cuestionario técnico de la guía.

Revisión de la Literatura

La memoria RAM es el almacenamiento temporal de variables durante la ejecución. La dirección hexadecimal es la representación en base 16 con prefijo *0x*, compacta y alineada a potencias de dos. El puntero es la variable que guarda la dirección de otra variable; el operador de dirección devuelve la ubicación física y el operador de desreferenciación accede o modifica el contenido del casillero apuntado (Stroustrup, 2013; cplusplus.com, 2024; Microsoft, 2024). La distinción entre dirección y contenido es el núcleo conceptual de la actividad.

>>TABLE: Expresión|Qué es|Qué imprime (ejemplo) p|Puntero (dirección)|0x61fe14 *p|Contenido apuntado|21 &edad;|Dirección de edad|0x61fe14 edad|Variable directa|21

Expresión	Qué es	Qué imprime (ejemplo)
p	Puntero (dirección)	0x61fe14
*p	Contenido apuntado	21
&edad;	Dirección de edad	0x61fe14
edad	Variable directa	21

Nota. Diferencia entre puntero, contenido y dirección.

Método

El código en *src/main.cpp* define *int edad = 0; int* p = &edad;*. La entrada se realiza con *cin >> *p*, donde el asterisco es obligatorio. La decisión se evalúa con *if (*p >= 18)* sobre el contenido, no sobre la dirección. La salida muestra **p* y *p* en hexadecimal. La estructura */cadenaspunteros* contiene *src/*, *docs/* y *capturas/*. La compilación se ejecutó con *g++ src/main.cpp -o src/cadenaspunteros.exe*. Las capturas incluyen la ejecución con 16 y 21 años y el esquema de memoria RAM. Las pruebas contemplaron el caso límite de 18 años (aprobado) y 17 (rechazado), verificando que la dirección hexadecimal varía entre

ejecuciones por la aleatorización del espacio de direcciones.

Resultados

Con 21 años el sistema informó ACCESO APROBADO, edad 21 y dirección *0x61fe14*. Con 16 años informó ACCESO RECHAZADO con la misma lógica. El programa no distingue si el dato se ingresó de forma directa o mediante puntero, dado que ambos escriben en el mismo casillero. La Figura 1 muestra la salida con ambas ramas y la dirección hexadecimal; la Figura 2 presenta el esquema de casilleros RAM, puntero y operadores.

Nota. Figura 1. Ejecución mostrando ramas APROBADO y RECHAZADO y valor hexadecimal (capturas/ejecucion_punteros.png). Figura 2. Esquema de casilleros RAM, puntero y operadores (capturas/esquema_ram.png).

Desarrollo Teórico: ¿Qué es un Puntero y la Desreferenciación?

La guía de la Actividad 4 introduce esta pregunta como base conceptual. Un puntero es una variable especial que no almacena un valor de aplicación —como una edad o un nombre—, sino la dirección de memoria física donde está alojada otra variable. Si la memoria RAM se visualiza como una hilera de casilleros numerados, cada casillero tiene un número de casillero —la dirección hexadecimal, por ejemplo *0x61fe14*— y un contenido —el valor guardado, por ejemplo *21*—. La variable *edad* ocupa un casillero; el puntero *p* guarda el número de ese casillero. En el código, *int* p = &edad;* significa: declara *p* como puntero a entero y asígnale la dirección de *edad* mediante el operador de dirección &.

Desreferenciar es la operación inversa: usar el operador asterisco para abrir el casillero apuntado y leer o modificar su contenido. La expresión **p* significa: accede al contenido del casillero cuya dirección está en *p*. Por ello, *cin >> *p;* guarda el número ingresado directamente en el casillero de *edad*, mientras que *cout << *p;* lee ese contenido. Sin desreferenciación, *p* es solo la dirección; con desreferenciación, **p* es el valor. Esta distinción —dirección frente a contenido— es el fundamento de todo el trabajo con memoria en C y C++ y la base para arreglos, pasaje por referencia y estructuras dinámicas

(Stroustrup, 2013; cplusplus.com, 2024).

Cuestionario Técnico de Control

A continuación se responden las tres preguntas técnicas de la guía.

Pregunta 1 — Diferencia entre & y *

Enunciado: ¿Qué valor imprime en pantalla la instrucción *cout* << *p;* comparada con *cout* << *p;?

Respuesta: La instrucción *cout* << *p;* imprime la dirección hexadecimal donde reside la variable *edad* —el "dónde"—, por ejemplo *0x61fe14*. Es el número de casillero en la memoria RAM. La instrucción *cout* << *p; desreferencia el puntero y accede al contenido de ese casillero —el "qué"—, por ejemplo *21*, que es el valor de *edad*. La primera muestra el contenedor; la segunda, el contenido. Confundir ambas es el error más frecuente al iniciarse con punteros y explica por qué en el programa se usa *p para evaluar la condición de acceso y *p* para mostrar la dirección física.

Pregunta 2 — Error Crítico al Guardar Entrada

Enunciado: ¿Por qué es obligatorio desreferenciar el puntero (*p) al usar *cin* para modificar el valor de la variable original?

Respuesta: Porque *cin* debe escribir en un entero y *p* no es un entero sino una dirección (*int**). La expresión *cin >> p;* sin asterisco intenta sobrescribir la dirección misma —el número de casillero— con un valor de aplicación, lo que produce error de tipos en compilación y, si se forzara, corrupción de la tabla de direcciones y fallo de segmentación. La expresión *cin >> *p;* desreferencia y escribe en el entero apuntado, es decir, en el casillero de *edad*. El asterisco es el operador que abre el casillero; sin él, se ataca el rótulo del casillero en lugar de su contenido. Por ello la guía enfatiza *cin >> *p;* como forma correcta y documenta que omitir el asterisco es error grave.

Pregunta 3 — Formato Hexadecimal